

EECE 7205: Fundamentals of Computer Engineering

Fall 2026

Lab Assignment #1:

Student Name	Hritvik Kuppa
NUID	003947137
Email	kuppa.h@northeastern.edu
Instructor	Dr. Fatema Nafa
Due Date	24 Sept 2026

Submission Checklist (must match syllabus requirements)

- Submit **one PDF** generated from this Overleaf report.
- Include **clear screenshots** for each task (text must be readable).
- Include your **GitHub profile link** and **Overleaf project link** in the report.
- File name: `Assignment01_LastName.pdf`

How to complete this assignment

1. Complete each task below in order.
 2. For each task, include the required evidence (screenshots, links, and short notes).
 3. When you paste a link, ensure it opens correctly (no private/local-only links unless instructed).
-

Problem 1

1. Understand the Problem

- **Goal:** Given a set of intervals, sort them by their starting value and merge overlapping intervals.
- **Input:**
 - The number of intervals.
 - Each interval has a start and end value.
- **Output:** Sorted intervals with no overlaps.
- **Edge cases:**
 - No intervals overlap.
 - All intervals overlap.

2. Design the Solution

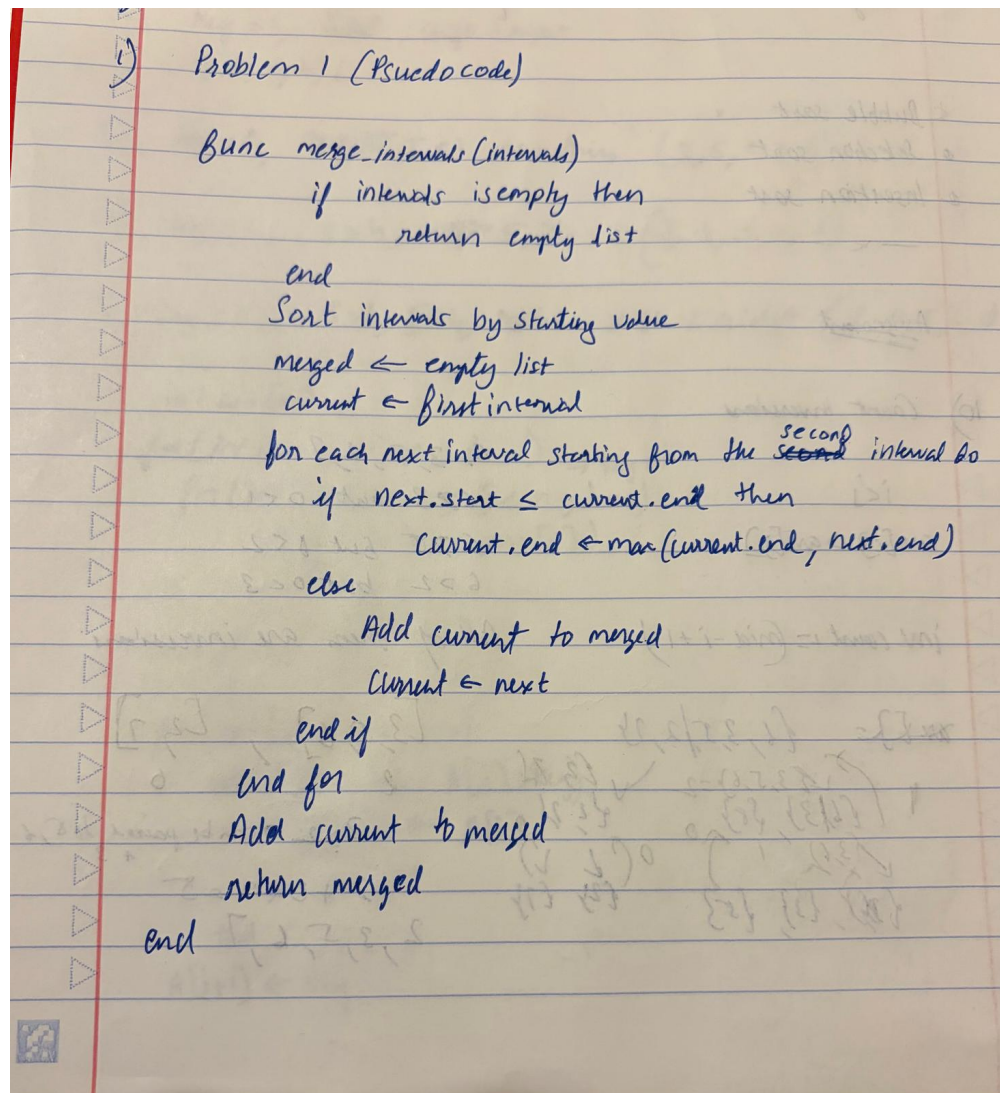
- **Algorithm:**

1. Sort the intervals by their starting values using Bubble Sort.
2. Check each interval with the next interval for an overlap.
3. If an overlap exists, merge the intervals by updating the end value.
4. If there is no overlap, keep the current interval and move to the next one.

- **Data Structure:**

A 2D array is used because each interval contains two values: a start and an end. Each row represents one interval.

3. Write Pseudocode



```
1) Problem 1 (Pseudocode)

func merge_intervals(intervals)
    if intervals is empty then
        return empty list
    end
    Sort intervals by starting value
    merged ← empty list
    current ← first interval
    for each next interval starting from the second second interval do
        if next.start ≤ current.end then
            current.end ← max(current.end, next.end)
        else
            Add current to merged
            current ← next
        end if
    end for
    Add current to merged
    return merged
end
```

Figure 1: Pseudocode for Problem 1

4. Explain Correctness

- The algorithm takes in the number of intervals, and then the intervals. The algorithm first sorts the interval by their starting values. This way we can check the end value of the current interval with the start value of the next interval, if the end value is greater there's an overlap and it can be merged.
- If the starting values of two intervals, they can be merged normally.

5. Analyze Complexity

Operation	Frequency
Check if intervals empty	1
Sort using bubble sort (two for loops)	n^2
Set merged to empty	1
Set current to first interval	1
Check each next interval	$n - 1$
Check overlap condition	$n - 1$
Merging overlapping intervals	$n - 1$
Add current interval to merged	n

$$T(n) = 1 + n^2 + 1 + 1 + (n - 1) + (n - 1) + (n - 1) + n$$

$$T(n) = n^2 + 4n$$

$$T(n) = O(n^2)$$

6. Implement in C++

```
int in;
int temp; //temp variable for sorting

cout << "Enter number of intervals: ";

//Asking the user to enter the number of intervals since arrays are static
//and have a fixed size, so I am asking to enter the number of intervals
cin >> in;

int interval[in][2];

//Entering the intervals
for (int i=0; i<in; i++){
    cout << "Enter the start and end of interval " << i+1 << ": ";

    //The i+1 is to basically show the interval number they are entering,
    //like interval 1, interval 2 etc..
    cin >> interval[i][0] >> interval[i][1];
}
```

```

//Sorting - Bubble sort with a temp variable
for (int i=0; i<in-1; i++){
    for (int j=0; j<in-1; j++){
        if (interval[j][0] > interval[j+1][0]){

            //checking if the value in the previous interval is greater or not
            temp = interval[j][0];
            interval[j][0] = interval[j+1][0];
            interval[j+1][0] = temp;

            //This is for the second value in the interval
            temp = interval[j][1];
            interval[j][1] = interval[j+1][1];
            interval[j+1][1] = temp;
        }
    }
}

//To check for overlaps and merge them
int merged = 0;

for(int i=1; i<in; i++){

    //If the current merged interval overlaps with the next interval,
    //then it keeps the larger ending value
    if (interval[merged][1] >= interval[i][0]){
        interval[merged][1] =
            max(interval[merged][1], interval[i][1]);
    }
    else{
        merged++; //moving to the next position

        //copying the current interval into next merged position
        interval[merged][0] = interval[i][0];
        interval[merged][1] = interval[i][1];
    }
}

//To print out the intervals
for (int i=0; i<=merged; i++){
    cout << "[" << interval[i][0] << "," << interval[i][1] << "];"

    if (i != merged) {
        cout << ",";
    }
}

```

7. Test the Solution

Normal case:

```
PS C:\Users\hritv\Desktop\NEU\FOCE> cd "c:\Users\hritv\Desktop\NEU\FOCE"
Choose a problem from 1-10: 1
Enter number of intervals: 5
Enter the start and end of interval 1: 1 3
Enter the start and end of interval 2: 2 6
Enter the start and end of interval 3: 8 10
Enter the start and end of interval 4: 9 12
Enter the start and end of interval 5: 15 18
[1,6],[8,12],[15,18]
```

Figure 2: Normal case test for Problem 1

Edge case:

```
Choose a problem from 1-10: 1
Enter number of intervals: 4
Enter the start and end of interval 1: 1 3
Enter the start and end of interval 2: 2 5
Enter the start and end of interval 3: 4 7
Enter the start and end of interval 4: 6 10
[1,10]
```

Figure 3: Edge case test for Problem 1

Problem 2

1. Understand the Problem

- **Goal:** Sort an array containing only 0s, 1s, and 2s so that all 0s come first, followed by all 1s, and then all 2s.
- **Input:**
 - An integer n representing the number of elements.
 - An array containing only 0s, 1s, and 2s.
- **Output:** A sorted array containing all 0s first, then all 1s, and then all 2s.
- **Edge cases:**
 - Already sorted array.
 - Reverse sorted array.
 - Empty array.

2. Design the Solution

- **Algorithm:**

Two methods are used: Counting Based and Dutch National Flag (DNF).

- **Counting Based:**

1. Count the number of 0s, 1s, and 2s in the array.
2. Rewrite the array using the counts, placing all 0s first, then 1s, and then 2s.

- **DNF Algorithm:**

1. Use three variables: low, mid, and high.
2. Values before low are 0s.
3. Values between low and mid are 1s.
4. Values between mid and high are unknown.
5. Values after high are 2s.
6. Move the elements into their correct regions until the unknown region is empty.

- **Data Structure:**

An array is used to store the 0s, 1s, and 2s.

3. Write Pseudocode

Counting Based

Problem 2

Counting

func sort-by-count (arr)

count-0 $\leftarrow$ 0

count-1 $\leftarrow$ 1

count-2 $\leftarrow$ 2

for each element in arr do

if element = 0 then

count-0 $\leftarrow$ count-0 + 1

if element = 1 then

count-1 $\leftarrow$ count-1 + 1

else

count-2 $\leftarrow$ count-2 + 1

end if

end for

index $\leftarrow$ 0

for i $\in$ 1 to count-0 do

arr[index] $\leftarrow$ 0

index $\leftarrow$ index + 1

end for

Figure 4: Counting-based pseudocode

```

for i ← 1 to count-1 do
    arr[index] ← 1
    index ← index + 1
end for

for i ← 1 to count-2 do
    arr[index] ← 2
    index ← index + 1
end for
return arr
end function

```

DNF Based

B) DNF Algo

```

func DNF(arr)
    low ← 0
    mid ← 0
    high ← n-1
    while mid ≤ high do
        if arr[mid] = 0 then
            swap(arr[low], arr[mid])
            low ← low + 1
            mid ← mid + 1
        end if
    end while

```

Figure 5: DNF-based pseudocode

```

else if arr[mid] = 1 then
    mid ++
else
    swap(arr[mid], arr[high])
    high --
end if
end while
end func

```

4. Explain Correctness

Counting

- After the input is taken, the number of 0s, 1s and 2s are counted in the array. The array is then written with the count of 0s, 1s and 2s, and the array is sorted.
- Example if there are three 0s, three 1s and two twos. The array is going to be re-written as 0,0,0,1,1,1,2,2.

DNF

- The algorithm consists of three regions; low, mid and high.
- 0 to low -1 consists of zeros.
- Low to mid -1 consists of ones.
- Mid to high are unknown.
- High +1 to n-1 consists of twos.
- While mid < high, the following operations happen.
- If the mid element is 0, it's moved to zero region or the low region.

- If the mid element is 1, it stays right there because it's in the correct place.
- If the mid element is 2, it's moved to 2 region or the high region.
- There can be multiple 0s, 1s and 2s so duplicates are naturally handled.

5. Analyze Complexity

Counting

Operation	Freq
Initializing counters	1
Checking each element	n
Incrementing a counter	n
Initializing index	1
0s	n
1s	n
2s	n
return arr	1

Total time complexity $T(n) = 5n + 2$

$$T(n) = O(n)$$

DNF

Operation	Freq
Initializing low,mid,high	1
Checking if $\text{mid} \leq \text{high}$	$N + 1$
Checking the current value	N
Swap/increment/decrement	N
Return arr	1

Time Complexity $T(n) = 3n + 3$

$$T(n) = O(n)$$

6. Implement in C++

```
int choice;
int n;

cout << "Enter the number of elements: ";
cin >> n;

vector<int> arr(n);

for (int i=0; i<n; i++){
```

```

        cout << "Enter element " << i+1 << ": ";
        cin >> arr[i];
    }

    cout << "Enter the number to select the method "
        << "(1 = Counting based, 2 = One pass DNF Algorithm): ";
    cin >> choice;

    //A menu to choose either Counting based or DNF
    switch(choice){

        case 1: {

            //Counting based
            int count_0 = 0;
            int count_1 = 0;
            int count_2 = 0;

            //Checking for 0,1,2 in the array
            //The count of 0 increases whenever it finds 0 in the array
            //and similar for 1,2
            for (int i=0; i<n; i++){
                if(arr[i] == 0)
                    count_0++;
                else if(arr[i] == 1)
                    count_1++;
                else
                    count_2++;
            }

            //Replacing the elements based on the count of 0s, 1s and 2s.
            int index = 0;

            for (int i=0; i<count_0; i++){
                arr[index++] = 0;
            }

            for (int i=0; i<count_1; i++){
                arr[index++] = 1;
            }

            for (int i=0; i<count_2; i++){
                arr[index++] = 2;
            }

            for (int i=0; i<n; i++){
                cout << arr[i] << ",";
            }
        }
    }

```

```

        break;
    }

    case 2: {

        //One Pass DNF

        //Initializing mid, high and low values.
        int mid = 0;
        int high = n-1;
        int low = 0;
        int temp;

        while(mid <= high){

            if (arr[mid] == 0){

                //If the middle value is 0, swap the value with the low
                //and increment mid and low
                temp = arr[low];
                arr[low] = arr[mid];
                arr[mid] = temp;

                mid++;
                low++;
            }

            //If arr[mid] is 1, no need to swap anything
            //since that's where 1 should be
            else if (arr[mid] == 1)
                mid++;

            else{

                //When value = 2, swap with high and decrement high
                temp = arr[high];
                arr[high] = arr[mid];
                arr[mid] = temp;

                high--;
            }
        }

        //Printing the array
        for (int i=0; i<n; i++){
            cout << arr[i] << ",";
        }
    }
}

```

```

    }

    break;
}

```

7. Test the Solution

Normal case (Counting):

```

Choose a problem from 1-10: 2
Enter the number of elements: 6
Enter element 1: 2
Enter element 2: 0
Enter element 3: 2
Enter element 4: 1
Enter element 5: 1
Enter element 6: 0
Enter the number to select the method (1 = Counting based, 2 = One pass DNF Algorithm): 1
0,0,1,1,2,2,

```

Figure 6: Normal case using the Counting method

Normal case (DNF):

```

PS C:\Users\hritv\Desktop\NEU\FOCE> cd "c:\Users\hritv\Desktop\NEU\FOCE\" ; if ($?) { g++ Assignment
Choose a problem from 1-10: 2
Enter the number of elements: 6
Enter element 1: 2
Enter element 2: 0
Enter element 3: 1
Enter element 4: 2
Enter element 5: 1
Enter element 6: 1
Enter the number to select the method (1 = Counting based, 2 = One pass DNF Algorithm): 2
0,1,1,1,2,2,
PS C:\Users\hritv\Desktop\NEU\FOCE>

```

Figure 7: Normal case using the DNF method

Problem 3

1. Understand the Problem

- **Goal:** Given a vector of non-negative integers, arrange them so that their concatenation forms the largest possible number.
- **Input:**
 - Number of elements n
 - n non-negative integers
- **Output:** The largest possible number formed by concatenating the integers.
- **Edge cases:**
 - One number
 - Repeated numbers

- Numbers such as 2, 20, and 200
- All numbers are zero

2. Design the Solution

- The algorithm converts all the integers into strings.
- Two numbers a and b are compared by checking $a + b$ and $b + a$.
- If $a + b < b + a$, the two strings are swapped.
- Bubble Sort is used to repeatedly compare adjacent strings using this custom comparison.
- After sorting, the strings are concatenated to form the largest possible number.
- **Data Structure:** A vector of strings is used because the input values need to be stored and swapped during sorting.

3. Write Pseudocode

Problem - 3

```

func largest-num (n)
    Convert num to string
    for i ← 0 to n-2 do
        for j ← 0 to n-2 do
            if (string[j] + string[j+1]) < (string[j+1] + string[j]) then
                swap(string[j] and string[j+1])
            and if
        end for
    end for
    result ← empty string
    return result
end func
  
```

Figure 8: Pseudocode for Problem 3

4. Explain Correctness

- The algorithm first concatenates two adjacent numbers a and b like $a+b$ and $b+a$.

- Then it compares the two values and if b+a is greater, b comes before a.
- Example let a = 4, b = 45 be two adjacent numbers.
 1. Concatenating the two numbers a+ b we get 445 and for b+a, we get 454.
 2. Since $445 < 454$, the numbers become 45, 4.
 3. Hence the correct order is 45, 4.

5. Analyze Complexity

Operation	Freq
Convert num to str	n
Initializing outer loop	1
Checking Outer for loop	$N - 1$
Initializing in inner loop	n
Checking Inner for loop	N^2
Increment inner loop	$N(n - 1)$
Comparing a+b < b+a	$N(n - 1)$
Concatenating the result	n

$$T(n) = 1 + n + (n - 1) + n + n^2 + n(n - 1) + n(n - 1) + n$$

$$T(n) = 3n^2 + 2n$$

$$T(n) = O(n^2)$$

6. Implement in C++

```
int n;

cout << "Enter a number of elements: ";
cin >> n;

vector<int> arr(n);
vector<string> string_arr;

string temp;

for (int i=0; i<n; i++){
    cout << "Enter the non-negative integer "
         << i+1 << ": ";
    cin >> arr[i];

    string_arr.push_back(to_string(arr[i]));
}

for (int i=0; i<n-1; i++){
```

```

for (int j=0; j<n-1; j++){

    if ((string_arr[j] + string_arr[j+1]) <
        (string_arr[j+1] + string_arr[j])){

        temp = string_arr[j];
        string_arr[j] = string_arr[j+1];
        string_arr[j+1] = temp;
    }
}

string result = "";

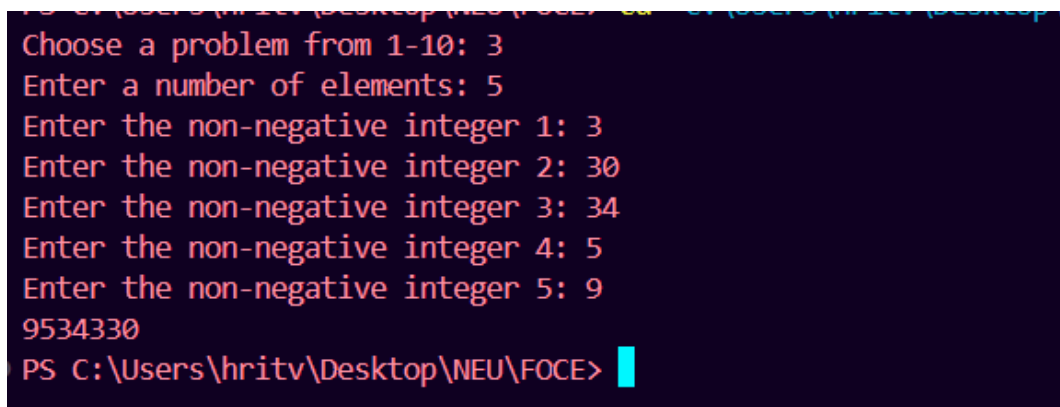
for (int i=0; i<n; i++){
    result += string_arr[i];
}

cout << result;

```

7. Test the Solution

Normal case:



```

PS C:\Users\hritv\Desktop\NEU\FOCE> Choose a problem from 1-10: 3
Enter a number of elements: 5
Enter the non-negative integer 1: 3
Enter the non-negative integer 2: 30
Enter the non-negative integer 3: 34
Enter the non-negative integer 4: 5
Enter the non-negative integer 5: 9
9534330
PS C:\Users\hritv\Desktop\NEU\FOCE>

```

Figure 9: Normal case test for Problem 3

Edge case:

```
PS C:\Users\hritv\Desktop\NEU\FOCE> cd "c:\Users\hri
Choose a problem from 1-10: 3
Enter a number of elements: 5
Enter the non-negative integer 1: 0
Enter the non-negative integer 2: 0
Enter the non-negative integer 3: 0
Enter the non-negative integer 4: 0
Enter the non-negative integer 5: 0
00000
PS C:\Users\hritv\Desktop\NEU\FOCE>
```

Figure 10: Edge case test for Problem 3

Problem 4

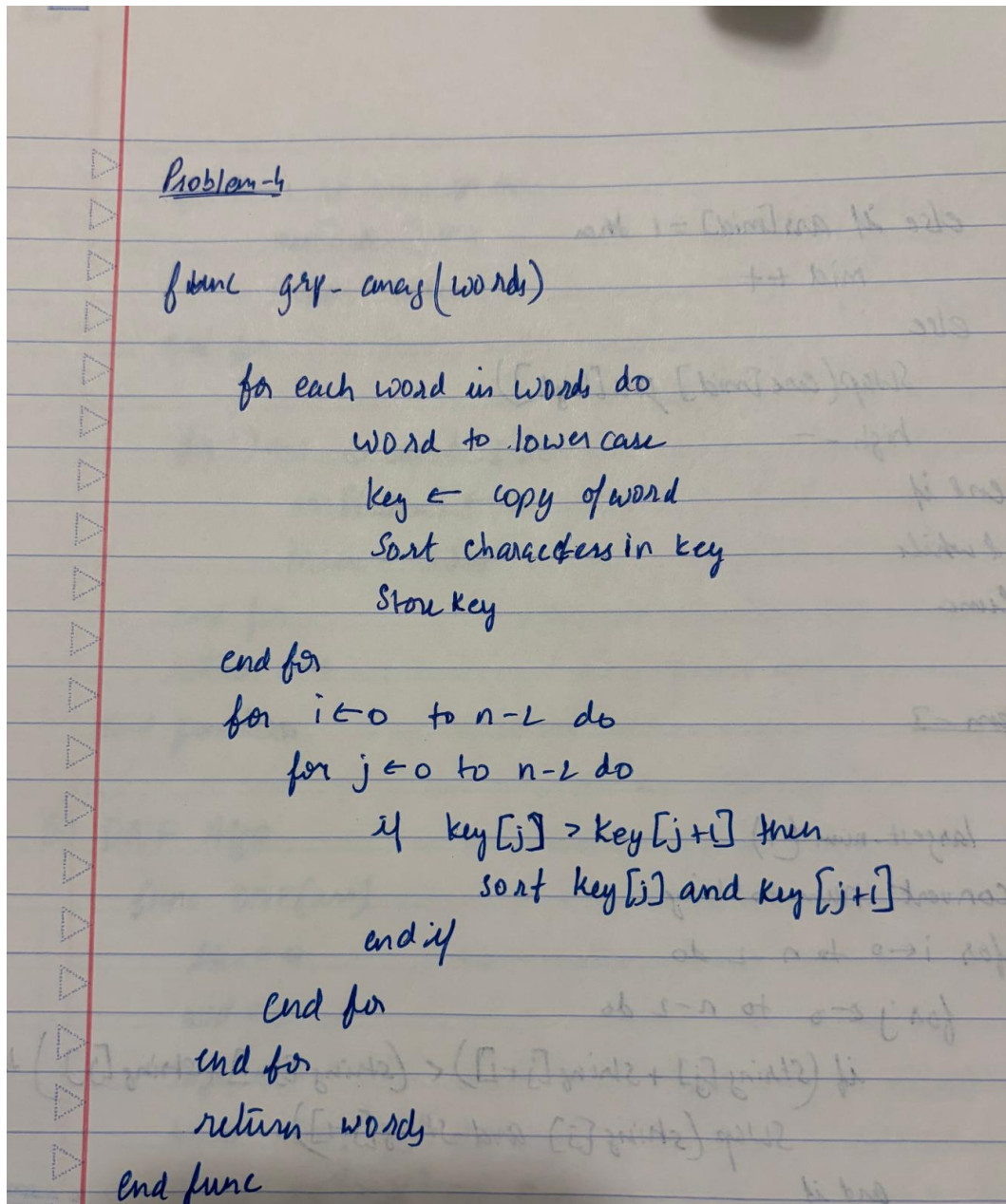
1. Understand the Problem

- **Goal:** Given a vector of strings, group the anagrams together.
- **Input:**
 - Number of words n
 - n words
- **Output:** The words arranged so that anagrams are placed together.
- **Edge cases:**
 - One word
 - No anagrams
 - All words are anagrams

2. Design the Solution

- Each word is converted to lowercase.
- A copy of each word is created and its characters are sorted.
- The sorted characters are used as the key for the word.
- Anagrams have the same key because they contain the same characters.
- Bubble Sort is then used to arrange the words according to their keys.
- **Data Structure:** Vectors are used to store the original words and their corresponding sorted keys.

3. Write Pseudocode



The image shows a handwritten pseudocode for a word sorting algorithm. The text is written in blue ink on lined paper. The pseudocode is as follows:

```
Problem-9  
  
func grr-ans(words)  
  
    for each word in words do  
        word to lower case  
        key ← copy of word  
        sort characters in key  
        store key  
    end for  
  
    for i ← 0 to n-2 do  
        for j ← 0 to n-2 do  
            if key[j] > key[j+1] then  
                sort key[j] and key[j+1]  
            end if  
        end for  
    end for  
  
    return words  
end func
```

Figure 11: Normal case test for Problem 9

- Convert every word to lowercase.
- Create a sorted-character key for every word.
- Compare the keys using Bubble Sort.
- If the key on the left is greater than the key on the right, swap both the keys and their corresponding words.
- Print the resulting words.

4. Explain Correctness

- The algorithm takes in n words as input.
- Sorts characters of each word to create a key.
- Words which are anagrams are going to have the same sorted characters and they are placed next to each other.
- The original input order is going to be retained since words with the same key are not swapped.

5. Analyze Complexity

Operation	Freq
Convert words to lowercase	N
Copy each word	N
Sort characters	N
Initialize outer loop	1
Outer loop	N
Increment outer loop	$n - 1$
Initialize inner loop	N
Inner loop	$n - 1$
Increment inner loop	$N(n - 1)$
Compare keys	$N(n - 1)$

$$T(n) = 3n^2$$

$$T(n) = O(n^2)$$

6. Implement in C++

```
int n;
string str;
vector<string> anag;

cout << "Enter number of words: ";
cin >> n;

string temp;

for (int i=0; i<n; i++){
    cout << "Enter a word: ";
    cin >> str;

    for (int j=0; j<str.length(); j++){
        str[j] = tolower(str[j]);
    }

    anag.push_back(str);
}
```

```

}

vector<string> temparr;

for (int i=0; i<n; i++){
    temp = anag[i];

    sort(temp.begin(), temp.end());

    temparr.push_back(temp);
}

for (int i=0; i<n-1; i++){
    for (int j=0; j<n-1; j++){

        if (temparr[j] > temparr[j+1]){

            swap(temparr[j], temparr[j+1]);
            swap(anag[j], anag[j+1]);
        }
    }
}

for (int i=0; i<n; i++){
    cout << anag[i] << " ";
}

```

7. Test the Solution

Normal case:

```
PS C:\Users\hritv\Desktop\NEU\FOCE> cd "c:\Users\h
Choose a problem from 1-10: 4
Enter number of words: 6
Enter a word: eat
Enter a word: tea
Enter a word: tan
Enter a word: ate
Enter a word: nat
Enter a word: bat
bat eat tea ate tan nat
PS C:\Users\hritv\Desktop\NEU\FOCE>
```

Figure 12: Normal case test for Problem 4

Edge case:

```
PS C:\Users\hritv\Desktop\NEU\FOCE> cd "c:\Users\hritv\Desktop\NEU\FO
Choose a problem from 1-10: 4
Enter number of words: 1
Enter a word: abc
abc
PS C:\Users\hritv\Desktop\NEU\FOCE> cd "c:\Users\hritv\Desktop\NEU\FO
```

Figure 13: Edge case test for Problem 4

Problem 5

1. Understand the Problem

- **Goal:** Determine whether one person can attend every meeting without any overlapping meetings.
- **Input:**
 - Number of intervals n
 - Start and end time of each interval
- **Output:** Whether the person can attend every meeting.
- **Edge cases:**
 - No meetings
 - One meeting
 - Meetings that end exactly when the next meeting starts
 - Overlapping meetings

2. Design the Solution

- The intervals are first sorted according to their starting times.
- Bubble Sort is used for sorting the intervals.
- After sorting, each interval is compared with the next interval.
- If the end of the current interval is greater than the start of the next interval, the meetings overlap.
- **Data Structure:** A two-dimensional array is used to store the start and end values of every interval.

3. Write Pseudocode

The image shows a handwritten pseudocode for a problem, likely 'Can Attend Every Meeting' (Problem 5). The code is written on lined paper with blue ink. It starts with a function definition 'func meetings(intervals)' and a loop 'for i = 0 to n-2 do'. Inside this loop, there is another loop 'for j = 0 to n-2 do' which contains an 'if' statement 'if (intervals[j].start > intervals[j+1].start) then'. This 'if' statement leads to a 'swap(j, j+1)' operation. After the inner loop, there is an 'end if' and 'end for' for the inner loop, followed by an 'end for' for the outer loop. Below this, a variable 'meeting' is set to 'true'. Then, another loop 'for j = 0 to n-2 do' is shown, containing an 'if' statement 'if intervals[j].end > intervals[j+1].start then'. This 'if' statement leads to 'meeting = false' and a 'break' statement. After the loop, there is an 'end if' and 'end for'. Then, an 'if' statement 'if meeting = true then' leads to 'print("Can attend every meeting")', and an 'else' statement leads to 'print("Cannot attend every meeting")'. Finally, there is an 'end if' and 'end func'.

```
func meetings(intervals)
    for i = 0 to n-2 do
        for j = 0 to n-2 do
            if (intervals[j].start > intervals[j+1].start) then
                swap(j, j+1)
            end if
        end for
    end for

    meeting ← true

    for j = 0 to n-2 do
        if intervals[j].end > intervals[j+1].start then
            meeting ← false
            break
        end if
    end for

    if meeting = true then
        print("Can attend every meeting")
    else
        print("Cannot attend every meeting")
    end if
end func
```

Figure 14: Pseudocode for Problem 5

4. Explain Correctness

- The algorithm first takes in the start and end of the intervals and then sorts them using bubble sort.
- To check if the person can attend every meeting, the algorithm checks for current interval end time and compare it with the next interval start time.
- If the current interval end time is greater, an overlap exists which means that person cannot attend the meeting.

- If the current interval is less than next interval start time, there's no overlap and that person can attend that meeting.
- It checks for every consecutive interval and if there's no overlap, that person can attend every meeting.

5. Analyze Complexity

Operation	Freq
Read each interval	n
Outer loop of bubble sort	$n - 1$
Inner loop of bubble sort	$n - 1 * n - 1$
Swap first values	$(n - 1)^2$
Swap second values	$(n - 1)^2$
Checking adjacent intervals	$n - 1$
Initializing meeting	1
Meeting = false	1

$$T(n) = n + (n - 1) + (n - 1)^2 + (n - 1)^2 + (n - 1)^2 + (n - 1) + 1 + 1$$

$$T(n) = O(n^2)$$

6. Implement in C++

```
int in;
int temp;

cout << "Enter number of intervals: ";
cin >> in;

int interval[in][2];

for (int i=0; i<in; i++){
    cout << "Enter the start and end of interval "
        << i+1 << ": ";

    cin >> interval[i][0] >> interval[i][1];
}

for (int i=0; i<in-1; i++){
    for (int j=0; j<in-1; j++){

        if (interval[j][0] > interval[j+1][0]){

            temp = interval[j][0];
            interval[j][0] = interval[j+1][0];
            interval[j+1][0] = temp;
```

```

        temp = interval[j][1];
        interval[j][1] = interval[j+1][1];
        interval[j+1][1] = temp;
    }
}

bool meeting = true;

for (int j=0; j<in-1; j++){

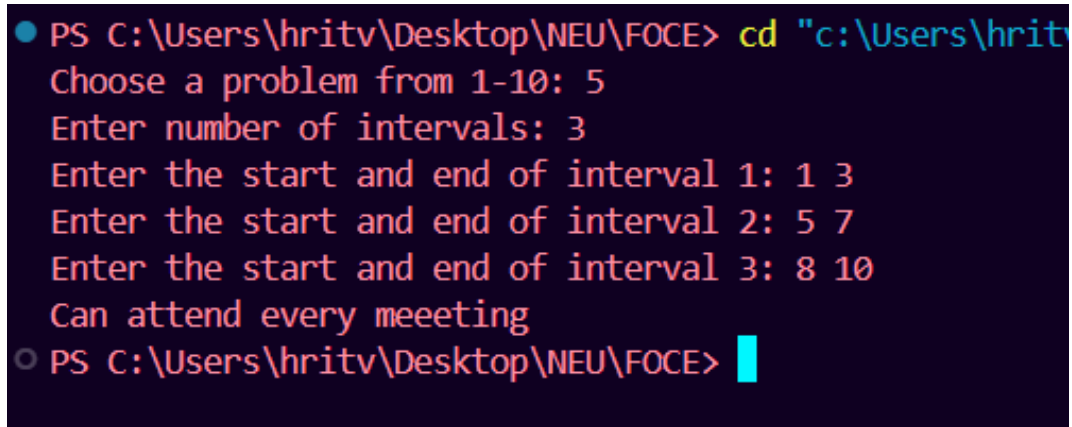
    if(interval[j][1] > interval[j+1][0]){
        meeting = false;
        break;
    }
}

if(meeting){
    cout << "Can attend every meeeting";
}
else{
    cout << "Cannot attend every meeting";
}

```

7. Test the Solution

Normal case:



```

● PS C:\Users\hritv\Desktop\NEU\FOCE> cd "c:\Users\hritv\Desktop\NEU\FOCE"
Choose a problem from 1-10: 5
Enter number of intervals: 3
Enter the start and end of interval 1: 1 3
Enter the start and end of interval 2: 5 7
Enter the start and end of interval 3: 8 10
Can attend every meeeting
○ PS C:\Users\hritv\Desktop\NEU\FOCE>

```

Figure 15: Normal case test for Problem 5

Edge case:

```

• Choose a problem from 1-10: 5
  Enter number of intervals: 2
  Enter the start and end of interval 1: 1 5
  Enter the start and end of interval 2: 5 7
  Can attend every meeting
• PS C:\Users\hritv\Desktop\NEU\FOCE>

```

Figure 16: Edge case test for Problem 5

Problem 6

1. Understand the Problem

- **Goal:** Given a set of points and an integer k , find the k points closest to the origin.
- **Input:**
 - Number of points n
 - Coordinates (x, y)
 - Integer k
- **Output:** The k points closest to the origin.
- **Edge cases:**
 - One point
 - The origin $(0, 0)$
 - Multiple points with the same distance
 - $k = n$

2. Design the Solution

- The distance of every point from the origin is calculated using:

$$d = \sqrt{x^2 + y^2}$$

- The points are sorted according to their distances.
- The program provides two sorting choices: normal Bubble Sort and Heap Sort.
- **Data Structure:** A two-dimensional array stores the coordinates and a separate array stores their distances.

3. Write Pseudocode

Part A: Finding the closest points

Problem 6

A) func closest-points(points, K)

```
for i ← 0 to n-1 do
    distance[i] ←  $\sqrt{x_i^2 + y_i^2}$ 
end for
for i ← 0 to n-2 do
    for j ← 0 to n-2 do
        if distance[i] > distance[j+1] then
            swap the coordinates, distances
        end if
    end for
end for
```

Figure 17: Pseudocode for finding the closest points

Part B: Heapify

```

8) func Heapify (coordinates, distance, i, n)
    while true do
        root ← i
        left ← 2*i + 1
        right ← 2*i + 2

        if left ≤ n & distance[left] > distance[root] then
            root ← left
        end if

        if right ≤ n & distance[right] > distance[root] then
            root ← right
        end if

        if root is not equal to i then
            swap coordinates[i] and coordinates[root]
            swap distance[i] and distance[root]
        end if
    end while
end func

```

Figure 18: Heapify pseudocode for Problem 6

4. Explain Correctness

- The algorithm first takes in n number of points and the k value from the user.
- Next, the distance is calculated using the formula $\sqrt{x^2 + y^2}$ from the origin.
- Then it's sorted based on the distances by both the sorting algorithms.
- The heap sorting builds a heap based on the distance and then sorts according to the distances. The sorting points are at the beginning.

- Finally, k points are printed out.

5. Analyze Complexity

a. Calculating the distance

Operation	Freq
Initializing coordinate array and distance array	1
Calculating x^2	N
Calculating y^2	N
Add $x^2 + y^2$	N
Sqrt $(x^2 + y^2)$	N

$$T(n) = 1 + 1 + N + N + N + N$$

$$T(n) = 2 + 5N$$

$$T(n) = O(N)$$

b. Normal Sorting

Operation	Freq
Outer loop	$n - 1$
Inner loop	$(n - 1)^2$
Comparing the distances	$(n - 1)^2$
Swapping x, y coordinates	$(n - 1)^2$
Swapping distance	$(n - 1)^2$
Printing k points	K

$$T(n) = (5n + 2) + (n - 1)^2 + (n - 1)^2 + (n - 1)^2 + (n - 1)^2 + k$$

$$T(n) = O(n^2)$$

c. Heap Sort

Operation	Freq
Calculating distances	N
Building initial heap	$n/2$
Heapify during build	N
Swap root with last element	$n - 1$
Each Extraction heapify	$n - 1 * \log n$

$$T(n) = O(n - 1 * \log(n))$$

$$T(n) = O(n \log n)$$

Normal Sorting:

- Bubble Sort uses two nested loops.

$$T(n) = O(n^2)$$

Heap Sort:

- Building the heap takes $O(n)$.
- Each extraction requires heapify, which takes $O(\log n)$.
- There are n extractions.

$$T(n) = O(n \log n)$$

6. Implement in C++

```
void heapify(int coord[][2], double dist[], int i, int n){
    while(true){
        int root = i;
        int left = (2*i) + 1;
        int right = (2*i) + 2;

        if (left < n && dist[left] > dist[root]){
            root = left;
        }

        if (right < n && dist[right] > dist[root]){
            root = right;
        }

        if(root != i){
            swap(coord[i], coord[root]);
            swap(dist[i], dist[root]);

            i = root;
        }
        else{
            break;
        }
    }
}

int k;
int n;
int temp;
```

```

int choice;

cout << "Enter number of pairs: ";
cin >> n;

int coord[n][2];
double dist[n];

for (int i=0; i<n; i++){
    cout << "Enter pair " << i+1 << ": ";
    cin >> coord[i][0] >> coord[i][1];
}

cout << "Enter the integer k: ";
cin >> k;

cout << "Enter your choice(1. Normal Sorting, 2. Heap Sort): ";
cin >> choice;

for (int i=0; i<n; i++){
    dist[i] = sqrt(pow(coord[i][0],2) +
                    pow(coord[i][1],2));
}

switch(choice){

    case 1:

        for (int i=0; i<n-1; i++){
            for (int j=0; j<n-1; j++){

                if (dist[j] > dist[j+1]){

                    temp = coord[j][0];
                    coord[j][0] = coord[j+1][0];
                    coord[j+1][0] = temp;

                    temp = coord[j][1];
                    coord[j][1] = coord[j+1][1];
                    coord[j+1][1] = temp;

                    double temp_dist = dist[j];
                    dist[j] = dist[j+1];
                    dist[j+1] = temp_dist;
                }
            }
        }
}

```

```

        for (int i=0; i<k; i++){
            cout << coord[i][0] << ","
                 << coord[i][1] << endl;
        }

        break;

case 2:

    for (int i=(n/2)-1; i>=0; i--){
        heapify(coord, dist, i, n);
    }

    for (int i=n-1; i>0; i--){

        swap(coord[0], coord[i]);
        swap(dist[0], dist[i]);

        heapify(coord, dist, 0, i);
    }

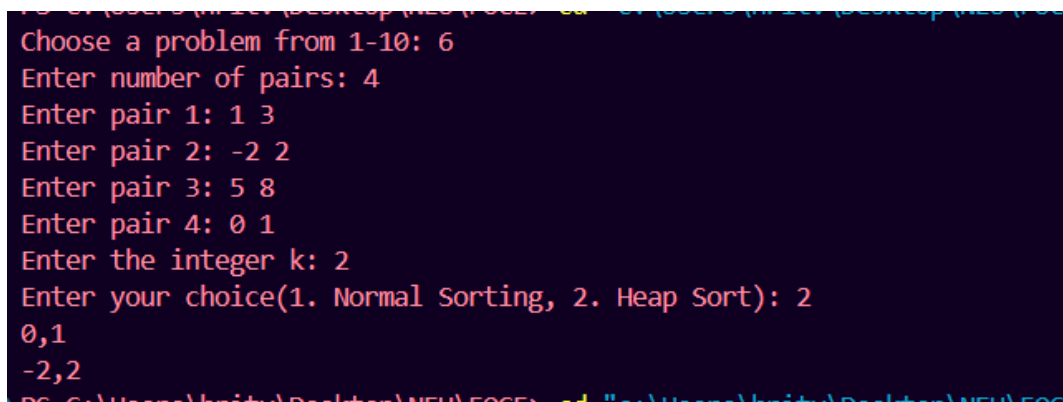
    for (int i=0; i<k; i++){
        cout << coord[i][0] << ","
             << coord[i][1] << endl;
    }

    break;
}

```

7. Test the Solution

Normal case:



```

PS C:\Users\hritu\Desktop\NEU\FECE> cd "C:\Users\hritu\Desktop\NEU\FECE"
Choose a problem from 1-10: 6
Enter number of pairs: 4
Enter pair 1: 1 3
Enter pair 2: -2 2
Enter pair 3: 5 8
Enter pair 4: 0 1
Enter the integer k: 2
Enter your choice(1. Normal Sorting, 2. Heap Sort): 2
0,1
-2,2
PS C:\Users\hritu\Desktop\NEU\FECE> cd "C:\Users\hritu\Desktop\NEU\FECE"

```

Figure 19: Normal case test for Problem 6

Edge case:

```

PS C:\Users\hritv\Desktop\NEU\FOCE> cd "c:\Users\hritv\Desktop\N
Choose a problem from 1-10: 6
Enter number of pairs: 1
Enter pair 1: 0 0
Enter the integer k: 1
Enter your choice(1. Normal Sorting, 2. Heap Sort): 2
0,0
PS C:\Users\hritv\Desktop\NEU\FOCE>

```

Figure 20: Edge case test for Problem 6

Problem 7

1. Understand the Problem

- **Goal:** Find the k most frequent values in an array.
- If two values have the same frequency, the smaller value should come first.
- **Input:**
 - Number of integers n
 - n integers
 - Integer k
- **Output:** The k most frequent values.
- **Edge cases:**
 - All values are the same
 - All values have the same frequency
 - $k = 1$

2. Design the Solution

- A map is used to count the frequency of every distinct value.
- The distinct values and their frequencies are then stored separately.
- Bubble Sort is used to sort the values.
- Values with a higher frequency are placed first.
- If frequencies are equal, the smaller value is placed first.
- **Data Structure:** A map is used for frequency counting and vectors are used for sorting.

3. Write Pseudocode

Problem 7

```
func k - freq(arr, k)
    count ← 0
    for i ← 0 to n-1 do
        count[arr[i]]++
    end for

    for each pair (value, freq) in count do
        store value in val
        store frequency in key
    end for

    for i ← 0 to m-2 do
        for j ← 0 to m-2 do
            if key[j] < key[j+1] then
                swap(key[j], key[j+1])
                swap(val[j], val[j+1])
            else if frequencies are equal & val[j] > val[j+1] then
                swap(key[j], key[j+1])
                swap(val[j], val[j+1])
            end if
        end for
    end for

end func
```

Figure 21: Pseudocode for Problem 7

4. Explain Correctness

- The algorithm first takes in the values and the integer k.
- It then counts the frequency of every distinct value in the array using a map.
- Next, the values are sorted using bubble sort in the decreasing order.
- If two values have the same frequency, the smaller value is placed first.
- Finally, it prints the first k values.

5. Analyze Complexity

Operation	Freq
Create array	1
Reading elements in array	N
Insert element in array	n
Storing distinct values and frequencies	M
Outer loop in sorting	$M - 1$
Inner loop in sorting	$(m - 1)^2$
Comparing frequencies	$(m - 1)^2$
Swapping frequencies and actual values	$(m - 1)^2$

$$T(n) = O(n^2)$$

6. Implement in C++

```
int n;
int k;

cout << "Enter number of integers: ";
cin >> n;

vector<int> arr(n);

for(int i=0; i<n; i++){
    cout << "Enter number in array: ";
    cin >> arr[i];
}

cout << "Enter k: ";
cin >> k;

map<int,int> count;

for(int i=0; i<n; i++){
    count[arr[i]]++;
}

int size = count.size();

vector<int> val(size);
vector<int> key(size);

int index = 0;

for (auto& p : count){
    val[index] = p.first;
    key[index] = p.second;
```

```

        index++;
    }

    for (int i=0; i<size-1; i++){
        for (int j=0; j<size-1; j++){

            if (key[j] < key[j+1]){

                swap(key[j], key[j+1]);
                swap(val[j], val[j+1]);
            }

            else if (key[j] == key[j+1] &&
                    val[j] > val[j+1]){

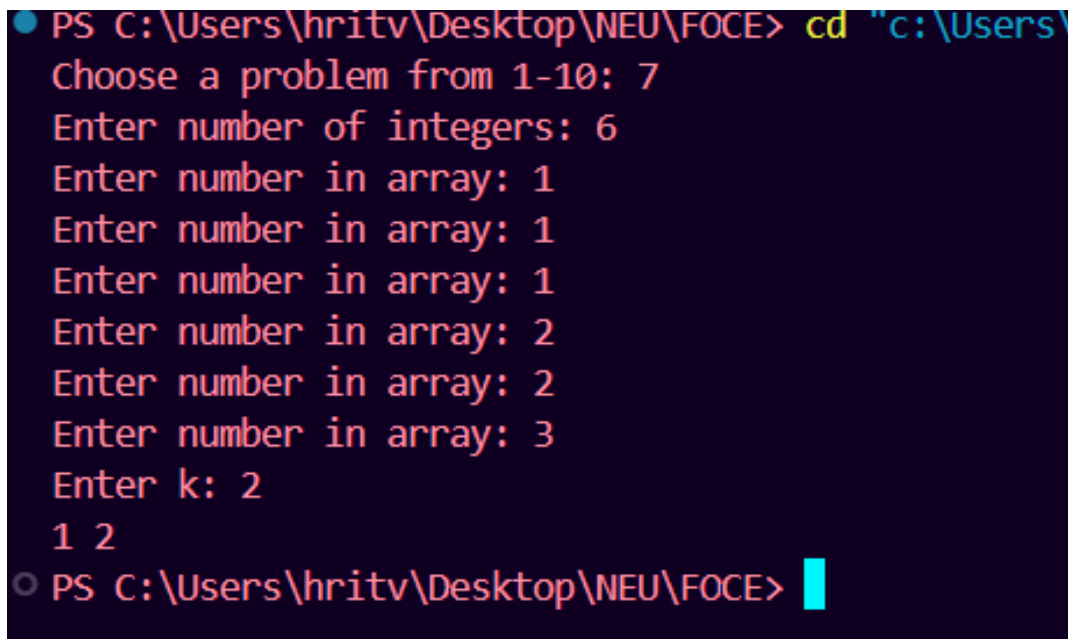
                swap(key[j], key[j+1]);
                swap(val[j], val[j+1]);
            }
        }
    }

    for (int i=0; i<k && i<size; i++){
        cout << val[i] << " ";
    }

```

7. Test the Solution

Normal case:



```

PS C:\Users\hritv\Desktop\NEU\FOCE> cd "c:\Users\hritv\Desktop\NEU\FOCE"
Choose a problem from 1-10: 7
Enter number of integers: 6
Enter number in array: 1
Enter number in array: 1
Enter number in array: 1
Enter number in array: 2
Enter number in array: 2
Enter number in array: 3
Enter k: 2
1 2
PS C:\Users\hritv\Desktop\NEU\FOCE>

```

Figure 22: Normal case test for Problem 7

Edge case:

```
● Choose a problem from 1-10: 7
  Enter number of integers: 4
  Enter number in array: 4
  Enter number in array: 4
  Enter number in array: 4
  Enter number in array: 4
  Enter k: 3
  4
○ PS C:\Users\hritv\Desktop\NEU\FOCE> █
```

Figure 23: Edge case test for Problem 7

Problem 8

1. Understand the Problem

- **Goal:** Given k sorted arrays, merge them into one sorted array.
- **Input:**
 - Number of arrays k
 - Size and elements of each sorted array
- **Output:** One sorted array containing all elements.
- **Edge cases:**
 - One array
 - Arrays with different sizes
 - Duplicate values
 - Empty arrays

2. Design the Solution

- Since all input arrays are already sorted, the algorithm keeps an index for each array.
- It searches for the smallest current value among the arrays.
- The smallest value is added to the result.
- The index of that array is then increased.
- This process continues until all arrays have been completely processed.
- **Data Structure:** A vector of vectors is used to store the sorted arrays, while another vector stores the current index of each array.

3. Write Pseudocode

The image shows a handwritten pseudocode for 'Problem 8' on a piece of lined paper. The text is written in dark ink and includes several corrections and annotations. The pseudocode describes a function 'Sorted-arrays' that takes an array 'arr' and a value 'k' as input. It uses a 'while' loop to repeatedly find the minimum element in the array, updating 'min_arr' and 'min_idx' as it goes. The process continues until 'min_arr' is no longer -1. The final sorted array is then printed.

```
Problem 8
Sorted-arrays(arr, k)
  while true do
    min_arr ← -1
    for i ← 0 to k-1 do
      if index[i] < size of arr[i] then
        if min_arr = -1 || arr[i][index[i]] < min then
          min ← arr[i][index[i]]
          min_idx ← i
        end if
      end if
    end for
    if min_arr = -1 then
      break
    end if
  end while
end func
```

Figure 24: Pseudocode for Problem 8

4. Explain Correctness

- The problem starts by taking the n number of arrays as input.
- There's a index for each array and it selects the smallest value among all the arrays repeatedly.
- After finding the element the index goes forward and since the input arrays are already sorted, the smallest element ensures that the array is sorted.
- All the duplicate values are included.
- Finally the sorted array is printed.

5. Analyze Complexity

Operation	Freq
Read elements in array	N
Create index vector	K
Initialize indices to 0	K
While loop	$N + 1$
Scan all arrays	$K * N$
Compare current element with minimum	$K * N$
Update minimum and minimum array	Upto N
Increment selected index	N
Check if all the arrays are finished	$N + 1$

$$T(N) = O(kN)$$

6. Implement in C++

```
int k;
int n;

cout << "Enter number of arrays: ";
cin >> k;

vector<vector<int>> arr(k);

for(int i=0; i<k; i++){

    cout << "Enter size of array " << i+1 << ": ";
    cin >> n;

    arr[i].resize(n);

    cout << "Enter elements in array ";

    for (int j=0; j<n; j++){
        cin >> arr[i][j];
    }
}

vector<int> index(k, 0);
vector<int> result;

while (true){

    int min;
    int min_arr = -1;

    for(int i=0; i<k; i++){
```

```

        if(index[i] < arr[i].size()){

            if (min_arr == -1 ||
                arr[i][index[i]] < min){

                min = arr[i][index[i]];
                min_arr = i;
            }
        }
    }

    if (min_arr == -1){
        break;
    }

    result.push_back(min);
    index[min_arr]++;
}

for (int val : result){
    cout << val << " ";
}

```

7. Test the Solution

Normal case:

```

● PS C:\Users\hritv\Desktop\NEU\FOCE> cd "c:\Users\hritv\Desktop\NEU\FOCE"
Choose a problem from 1-10: 8
Enter number of arrays: 3
Enter size of array: 1: 3
Enter elements in array 1
4
7
Enter size of array: 2: 3
Enter elements in array 2
5
8
Enter size of array: 3: 3
Enter elements in array 3
6
9
1 2 3 4 5 6 7 8 9
○ PS C:\Users\hritv\Desktop\NEU\FOCE>

```

Figure 25: Normal case test for Problem 8

Edge case:

```

● Choose a problem from 1-10: 8
Enter number of arrays: 2
Enter size of array: 1: 0
Enter elements in array 1
Enter size of array: 2: 3
Enter elements in array 2
2
3
1 2 3
○ PS C:\Users\hritv\Desktop\NEU\FOCE>

```

Figure 26: Edge case test for Problem 8

Problem 9

1. Understand the Problem

- **Goal:** Find the k th largest element in an unsorted array.
- **Input:**
 - Number of integers n
 - n integers

- Integer k
- **Output:** The k th largest element.
- **Edge cases:**
 - Duplicate values
 - $k = 1$
 - $k = n$
 - $k > n$
 - All values are equal

2. Design the Solution

- The array is sorted in descending order using Bubble Sort.
- Once sorted, the k th largest element is located at index $k - 1$.
- **Data Structure:** An array is used to store the integers.

3. Write Pseudocode

- Read the array and k .
- Check whether k exceeds the size of the array.
- Sort the array in descending order using Bubble Sort.
- Print the element at index $k - 1$.

4. Explain Correctness

- The algorithm first takes in the unsorted arrays and integer k for input.
- The algorithm is sorted in a descending order using bubble sort.
- Finally, $k-1$ value is printed for the k th largest element.

5. Analyze Complexity

Operation	Freq
Read number of elements	1
Read k	1
Checking if $n < k$	1
Reading elements in array	N
Outer loop in bubble sort	$n - 1$
Inner loop in bubble sort	$(n - 1)^2$
Comparing adjacent elements	$(n - 1)^2$
Create temp variable	$(n - 1)^2$
Assign $arr[j]$, $arr[j+1]$	$(n - 1)^2$
Printing k th largest element	1

$$T(n) = 1 + 1 + 1 + n + (n - 1) + (n - 1)^2 + (n - 1)^2 + (n - 1)^2 + (n - 1)^2 + 1$$

$$T(n) = O(n^2)$$

6. Implement in C++

```
int n;
int k;

cout << "Enter number of integers in array: ";
cin >> n;

int arr[n];

cout << "Enter k: ";
cin >> k;

if(n < k){
    cout << "Exceeds the size of the array";
    return;
}

for (int i=0; i<n; i++){
    cout << "Enter elements in array: ";
    cin >> arr[i];
}

for (int i=0; i<n-1; i++){
    for (int j=0; j<n-1; j++){

        if (arr[j] < arr[j+1]){

            int temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
    }
}

cout << arr[k-1] << " is the largest element";
```

7. Test the Solution

Normal case:

```
● Choose a problem from 1-10: 9
Enter number of integers in array: 6
Enter k: 2
Enter elements in array: 3
Enter elements in array: 2
Enter elements in array: 1
Enter elements in array: 5
Enter elements in array: 6
Enter elements in array: 4
5 is the largest element
```

Figure 27: Normal case test for Problem 9

Edge case:

```
● PS C:\Users\hritv\Desktop\NEU\FOCE> cd "c:
Choose a problem from 1-10: 9
Enter number of integers in array: 3
Enter k: 2
Enter elements in array: 3
Enter elements in array: 3
Enter elements in array: 3
3 is the largest element
○ PS C:\Users\hritv\Desktop\NEU\FOCE>
```

Figure 28: Edge case test for Problem 9

Problem 10

1. Understand the Problem

- **Goal:** Count the number of inversions in an array.
- An inversion is a pair (i, j) such that:

$$i < j$$

and

$$value[i] > value[j].$$

- **Input:**

- Number of integers n
- n integers

- **Output:** The total number of inversions.
- **Edge cases:**
 - One element
 - Empty array
 - Already sorted array
 - Duplicate values

2. Design the Solution

- Merge Sort is used to sort the array while counting inversions.
- The array is divided into two halves recursively.
- During merging, if an element from the right half is smaller than an element from the left half, it forms an inversion with all remaining elements in the left half.
- Therefore, the inversion count can be calculated efficiently during the merge process.
- **Data Structure:** A vector is used to store the input array and a temporary vector is used during merging.

3. Write Pseudocode

Problem 10

```

func count-inversions(arr, low, high)
    if low < high then
        mid ← low + (high - low) / 2
        left-count ← count-inversions(arr, low, mid)
        right-count ← count-inversions(arr, mid + 1, high)
        merge-count ← Merge(arr, low, mid, high)

        return left-count + right-count + merge-count
    end if
end count-inversion
  
```

Figure 29: Pseudocode for Problem 10

```

func Merge(arr, low, mid, high)

    i ← low
    j ← mid + 1
    inversion-count ← 0

    while i ≤ mid & j ≤ high do
        if arr[i] ≤ arr[j] then
            Add arr[i] to temp array
            i++
        else
            Add arr[j] to temp array
            j++
        end if
    end while

    while i ≤ mid do
        Add arr[i] to temp array
        i++
    end while

    while j ≤ high do
        Add arr[j] to temp array
        j++
    end while

return
end func

```

Figure 30: Pseudocode for Problem 10

4. Explain Correctness

- The algorithm first takes in the array.
- It uses merge sort to divide the array into smaller parts and sort them.
- While merging the array, it counts for inversions. If the element in the right half is smaller, it forms an inversion with all the elements in the left half.
- Finally it prints the count of inversions by adding the inversion from the left half and right half.

5. Analyze Complexity

a. Merging time complexity

Operation	Frequency
Initializing I, j inversion count	1
Comparing left elements and right elements	N
Add elements to temp	N
Increment I or j	n
Calc inversion count	N
Copy remaining left elements and right elements	N
Temp to array	N
Return inversion count	1

$$T(n) = O(n)$$

b. Sorting time complexity

Operation	Freq
Check if low < high	N
Calculating mid	N
Recursive left sort	$N/2$
Recursive right sort	$n/2$
Merging left and right	N
Adding all the inversion counts and returning	N

$$T(n) = 2T(n/2) + O(n)$$

Array is repeatedly divided in half until $n \log_2 n$

$$T(n) = n \log_2 n + n \log 2 \text{ (left + right)}$$

$$T(n) = O(n \log n)$$

Therefore,

$$T(n) = O(n \log n)$$

6. Implement in C++

```
int merge(vector<int> &arr, int low, int mid, int high){

    vector<int> temp;

    int i = low;
    int j = mid + 1;

    int inversion_count = 0;

    while(i <= mid && j <= high){

        if(arr[i] <= arr[j]){

            temp.push_back(arr[i]);
            i++;
        }
        else{

            temp.push_back(arr[j]);
            j++;

            inversion_count += (mid - i + 1);
        }
    }

    while(i <= mid){

        temp.push_back(arr[i]);
        i++;
    }

    while(j <= high){

        temp.push_back(arr[j]);
        j++;
    }

    for(int index=0; index<temp.size(); index++){
        arr[index+low] = temp[index];
    }

    return inversion_count;
}

int merge_sort(vector<int> &arr, int low, int high){
```

```

    if(low < high){

        int mid = low + (high-low)/2;

        int l_inversion_count =
            merge_sort(arr, low, mid);

        int r_inversion_count =
            merge_sort(arr, mid+1, high);

        int inversion_count =
            merge(arr, low, mid, high);

        return l_inversion_count +
            r_inversion_count +
            inversion_count;
    }

    return 0;
}

int n;

cout << "Enter number of integers in the array: ";
cin >> n;

vector<int> arr(n);

for (int i=0; i<n; i++){

    cout << "Enter integer in the array: ";
    cin >> arr[i];
}

int count = merge_sort(arr, 0, n-1);

cout << count;

```

7. Test the Solution

Normal case:

```
Choose a problem from 1-10: 10
Enter number of integers in the array: 5
Enter integer in the array: 2
Enter integer in the array: 4
Enter integer in the array: 1
Enter integer in the array: 3
Enter integer in the array: 5
3
```

Figure 31: Normal case test for Problem 9

Edge case:

```
PS C:\Users\hritv\Desktop\NEU\FOCE> cd "c:\Users\hr
Choose a problem from 1-10: 10
Enter number of integers in the array: 5
Enter integer in the array: 1
Enter integer in the array: 2
Enter integer in the array: 3
Enter integer in the array: 4
Enter integer in the array: 5
0
PS C:\Users\hritv\Desktop\NEU\FOCE>
```

Figure 32: Edge case test for Problem 9